

**UNIVERSIDAD ESTATAL PENÍNSULA DE
SANTA ELENA**

**FACULTAD DE SISTEMAS Y TELECOMUNICACIONES
CARRERA DE INGENIERIA EN SOFTWARE**



ASIGNATURA:
CALIDAD DE SOFTWARE

TEMA:
ENTREGABLE 2: ANÁLISIS DE RIESGOS Y REVISIÓN TÉCNICA DEL PROYECTO

ELABORADO POR:

- ANDY BRYAN ALEJANDRO VERA
- ALISSON YAMEL REYES RICARDO
- YANDRIS MIGUEL RIVERA TORRES

CURSO Y PARALELO:
SOFTWARE 6/1

DOCENTE:
ING. ANTHONY ABRAHAN PACHAY ESPINOZA

LA LIBERTAD – ECUADOR

ENTREGABLE 2: ANÁLISIS DE RIESGOS Y REVISIÓN TÉCNICA DEL PROYECTO

Objetivo del Entregable

Identificar riesgos potenciales que puedan afectar la calidad del sistema y ejecutar una revisión técnica temprana de los artefactos desarrollados, con el fin de detectar defectos, inconsistencias y oportunidades de mejora antes de la fase formal de pruebas.

El propósito es aplicar actividades preventivas de aseguramiento de calidad (QA Preventivo), reduciendo la probabilidad de defectos durante la implementación y fortaleciendo la calidad del producto desde etapas tempranas.

Ciclo de Vida del Aseguramiento de Calidad (SQE)

En este hito aún no se ejecutan pruebas. El equipo se encuentra en una fase de mitigación y revisión estática, simulando el comportamiento real de un equipo de Ingeniería de Calidad de Software mediante el siguiente flujo secuencial:

Planificación (E1) → Análisis de Riesgos y Revisión Técnica (E2) → Diseño de Pruebas (E3) → Ejecución (E4) → Métricas (E5) → Análisis de Código (E6) → Rendimiento (E7) → Auditoría Final (E8).

1. Estado Actual del Proyecto

Breve descripción diagnóstica del avance alcanzado en el desarrollo, especificando de forma transparente:

- **Funcionalidades implementadas:** Listado de componentes web que ya se encuentran codificados.

Backend – Laravel con API REST

Módulo	Componentes implementados
Autenticación	Login, logout, refresh de token, `/me` — JWT propio con tabla `sessions`, `JwtAuthenticate` middleware
Gestión de Incidencias	CRUD completo (`/api/incidents`), modelo con PostGIS `geom: Point(4326)`, soft deletes
Categorías	Jerarquía Tipo → Subtipo (`incident_categories`), validación de categoría hoja via trigger
Ubicaciones	Jerarquía País → Provincia → Ciudad con geometría PostGIS en tabla `locations`
Historial de Estados	Registro automático via trigger de base de datos (`trg_log_incident_status`)
Asignaciones	CRUD de asignación de responsables a incidencias (`assignments`)

Roles y Permisos	Tablas `permissions`, `role_permission` con RBAC — FKs con cascade delete
Usuarios	CRUD completo con rol asignado (`users`)
Organizaciones	CRUD (`organizations`)
Menús	Menús por rol de usuario (`menus`, `menu_permissions`)
Sesiones JWT	Tabla `sessions` con tokens revocables, índices parciales de optimización

Base de datos – PostgreSQL y PostGIS

- 19 migraciones ejecutadas respetando dependencias.
- PostGIS habilitado (`enable\_postgis`).
- 3 triggers automáticos: validación de categoría hoja, log de cambios de estado, asignación geográfica automática por `ST\_Contains`.
- Restricciones CHECK en tabla `incidents` para `status` y `priority`.
- Llaves foráneas explícitas con `cascadeOnDelete()` en permisos, roles y menús.

Frontend – SPA

- Página de login (`login.component.html`).
- Dashboard (`dashboard.component.html`).
- Formulario de creación de incidencias con mapa Leaflet + OpenStreetMap (`incidencias.form.component.html`).
- Listado de incidencias (`incidencias.index.component.html`).
- Layout con topbar y sidebar responsivo.
- **Funcionalidades pendientes:** Módulos o flujos que restan por construir.
- Frontend (módulos sin implementar)**
 - Gestión de estados de incidencias (UI)
 - Asignación de responsables (UI)
 - Módulo de comentarios (UI)
 - Notificaciones (UI)
 - Clasificación jerárquica Tipo/Subtipo (UI gestión)
 - Ubicaciones normalizadas (UI gestión)
 - Consultas, filtros y métricas (UI)
 - Gestión de roles y permisos (UI)
 - Gestión de usuarios (UI panel administrador)

Backend

- Sin middleware de rate-limiting en el endpoint de login (brute force no protegido)
- Sin políticas de complejidad de contraseñas (solo longitud mínima 8 caracteres)
- El formulario frontend incluye campos `titulo` y `descripcion` que no existen en la tabla `incidents` ni en `StoreIncidentRequest` — hay una

desalineación entre el contrato de API documentado y la implementación actual

- CRUD de comentarios
- CRUD de notificaciones

- **Estado general del proyecto:** Diagnóstico del hito técnico actual. El proyecto se encuentra en un 40-50% de avance. La capa de backend (API REST) está casi completa con todos los módulos de dominio implementados. La base de datos tiene la estructura completa con integridad referencial y lógica automatizada via triggers. El frontend está en etapas iniciales: únicamente los módulos de autenticación, dashboard e incidencias (crear/listar) han sido codificados. La mayoría de módulos de gestión aún no tienen interfaz de usuario.

2. Matriz de Riesgos de Calidad e Inspección OWASP

El equipo deberá identificar anomalías antes de que ocurran, clasificadas bajo tres vertientes:

- **Riesgos Funcionales:** Ej. Pérdida de comentarios de incidencias, duplicidad de reportes o flujo erróneo en el cambio de estados.

Riesgo Funcional	Descripción	Evidencia en Código
Pérdida de datos de incidencia	El formulario frontend captura campos `titulo` y `descripcion` que no existen en la tabla `incidents` — datos silenciosamente descartados al enviar	`StoreIncidentRequest.php` no valida `titulo`/`descripcion`; migración `create_incidents_table.php` no los define
Duplicidad de categorías hoja	Sin validación única en subtipo, podrían crearse subtipos duplicados	`IncidentCategoryController` sin constraint `unique` en nombre+parent_id
Flujo de estados inconsistente	El trigger `trg_log_incident_status` registra cambios pero no bloquea transiciones inválidas (ej. pasar de `resolved` a `pending`)	`log_incident_status()` en migración `create_incident_triggers.php`

- **Riesgos Técnicos:** Ej. Desalineación en la integración de APIs cartográficas, bloqueos en la base de datos o fallos de empaquetamiento en Docker.

Riesgo Funcional	Descripción	Evidencia en Código
Desalineación entre API y Frontend	El contrato documentado en CLAUDE.md define campos distintos a los	Campos `titulo`, `descripcion`, `telefono`, `tipo`, `subtipo` en formulario vs.

	implementados en `StoreIncidentRequest`	`incident_category_id`, `location_id`, `priority`, `geom` en request
Bloqueo geoespacial	La función `auto_assign_location()` hace un `ST_Contains` que puede fallar si `locations` no tiene geometrías cargadas	Migración `create_incident_triggers.php` sin datos semilla de polígonos geográficos
Imagen Docker con dependencias externas	El frontend llama Leaflet y Feather Icons desde CDN sin conectividad no funciona el mapa ni los íconos	`incidencias.form.component.html` líneas de Leaflet CDN

- **Riesgos de Seguridad (OWASP Evaluación Preliminar):** Revisión obligatoria del estado actual frente a vulnerabilidades básicas:

Vector de Riesgo OWASP	¿Existe riesgo?	Observación Inicial / Diagnóstico
Broken Access Control	Sí	El middleware `JwtAuthenticate` autentica al usuario pero no verifica que el `incident_id` en la URL pertenezca al usuario autenticado. Un reportero podría acceder a `GET /api/incidents/{id}` con el ID de otro usuario.
Authentication Failures	Sí	`AuthService::login()` no implementa throttling ni bloqueo tras intentos fallidos. La contraseña solo exige mínimo 8 caracteres (`StoreUserRequest`) sin política de complejidad (sin requerir mayúsculas, números o símbolos).
Injection (SQL/NoSQL/XSS)	No Evidenciado	Se utiliza el ORM Eloquent con parámetros ligados no se construyen consultas SQL concatenando strings. Los datos geoespaciales usan la librería `eloquent-spatial`.
Security Misconfiguration	Sí	El entorno de desarrollo mantiene `APP_DEBUG=true` en el archivo `.env`, lo que expone stack traces completos (rutas de archivos, variables de entorno) en respuestas de error HTTP.
Cryptographic Failures	Bajo riesgo	Las contraseñas se almacenan con `Hash::make()` (bcrypt por defecto en Laravel). Los JWT usan el secreto configurado en `.env`. El refresh token se guarda hashado en la BD.

3. Priorización de Riesgos (Matriz Probabilidad × Impacto)

Clasificación de los riesgos mediante la evaluación combinada de su frecuencia estimada y severidad técnica:

Riesgo Evaluado	Probabilidad	Impacto	Prioridad
Acceso no autorizado a incidencias ajenas (Broken Access Control)	Alta	Alta	Crítico
Pérdida de campos `titulo`/`descripcion`, desalineación entre API y Frontend	Alta	Alta	Crítico
Ataque de fuerza bruta al endpoint de login sin throttling	Media	Alta	Alta
Exposición de errores detallados en modo debug (Security Misconfiguration)	Alta	Media	Alta
Lentitud en consulta de filtrado geoespacial `ST_Contains` sin índice GIST	Media	Media	Media
Asignación automática de ubicación falla si no hay polígonos en BD	Baja	Alta	Media
Íconos y mapa no cargan sin conexión a Internet (dependencia CDN)	Baja	Media	Baja

4. Revisión Técnica Estructurada del Sistema

El núcleo del entregable consiste en someter los artefactos a una inspección minuciosa bajo cuatro dimensiones:

4.1. Revisión de Requisitos

Hallazgos:

- El requisito funcional documenta que una incidencia debe registrar titulo (max 100) y descripcion (max 500), pero la tabla incidents en base de datos no contiene estas columnas, y el StoreIncidentRequest no las valida. Existe una laguna entre el requisito documentado y el artefacto implementado.
- El flujo de cambio de estados (pendiente → en_proceso → resuelto) no tiene restricciones de transición definidas — cualquier cambio de estado es permitido, incluyendo retroceder de resuelto a pendiente.
- El requisito de "doble validación" (frontend + backend) se cumple parcialmente: el frontend valida titulo y descripcion pero el backend no los procesa.

Cobertura de requisitos implementados: ~70% de los módulos de backend tienen endpoint, ~20% del frontend está codificado.

4.2. Revisión del Modelo de Datos

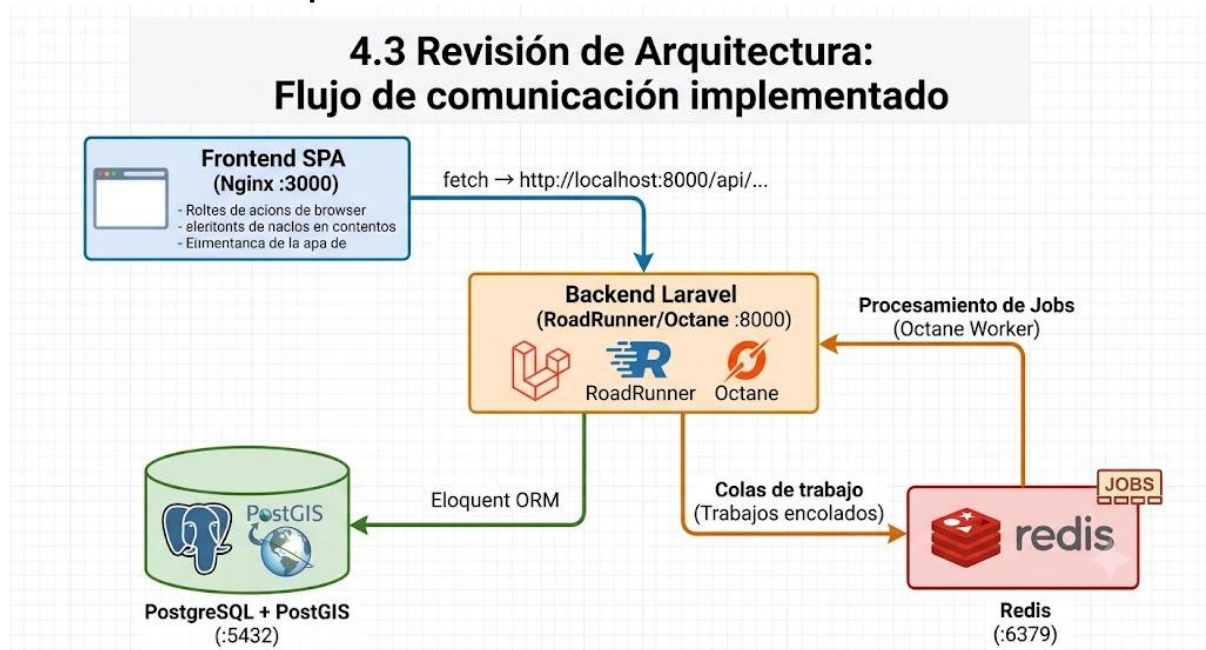
Fortalezas encontradas:

- Todas las tablas principales usan llaves foráneas explícitas con constrained() de Laravel.
- La tabla role_permission y menu_permissions tienen cascadeOnDelete() — eliminar un rol/permiso propaga correctamente.
- Soft deletes en entidades principales (incidents, users, incident_categories, organizations).
- Extensión PostGIS activada; columna geom de tipo geometry(Point, 4326) con SRID correcto.
- Restricciones CHECK en BD para valores de status y priority — integridad garantizada por motor.

Debilidades encontradas:

- La tabla incidents no tiene campos título ni descripción — no cumple con el payload documentado.
- La tabla status_history recibe datos del trigger con COALESCE(NEW.user_id, OLD.user_id) — si el campo user_id es NULL puede perder trazabilidad del actor que cambió el estado.
- No existe índice espacial GIST explícito en la columna geom de incidents para optimizar consultas geoespaciales de alto volumen.

4.3. Revisión de Arquitectura



Fortalezas:

- Arquitectura por dominios en /app/Domains/* — separación clara de responsabilidades.
- JWT con tabla de sesiones permite revocar tokens sin lista negra en memoria.
- Middleware JwtAuthenticate centraliza la autenticación — no hay rutas protegidas sin pasar por este filtro.

- Health check en GET /api/health facilita monitoreo en Docker.

Debilidades:

- El frontend llama directamente a http://localhost:8000 (hardcoded) — en despliegue producción esto falla.
- No existe configuración de CORS visible en bootstrap — puede causar errores en el navegador si frontend y backend están en dominios/puertos distintos.
- El endpoint /api/login no está bajo rate-limiting: no hay middleware throttle: aplicado en las rutas públicas.

4.4. Revisión Manual de Código Fuente

Organización de directorios: Estructura por dominios bien organizada (Domains/Auth, Domains/Incidents, etc.). Sin archivos huérfanos evidentes.

Estándares:

- Se usa declare(strict_types=1) en todos los archivos PHP revisados — buena práctica de tipado estricto.
- Los Form Requests tienen authorize() y rules() separados correctamente.
- Frontend usa Bootstrap 5 validation nativa con clases was-validated, is-invalid.

Problemas detectados:

- StoreUserRequest usa 'avatar'=> 'nullable|array' — tipo array no es almacenable directamente; falta conversión a JSON o tipo string.
- El trigger auto_assign_location() no registra en log si la ubicación no fue encontrada — falla silenciosa.
- En AuthService::login() se ejecuta User::where('email', \$email)->first() sin índice verificado en email — puede ser lento con muchos usuarios si no hay índice en ese campo.
- El código de StoreIncidentRequest no valida título, descripción, teléfono que el formulario envía — la API ignora estos campos sin error.

5. Lista de Verificación de Calidad (Checklist)

Ítem / Control de Calidad Analizado	¿Cumple formalmente?	Observación
CRUD operativo de gestión de incidencias georreferenciadas	SÍ	Endpoints POST/GET/PUT/DELETE /api/incidents implementados con coordenadas PostGIS
Validaciones aplicadas en el Frontend (Formatos, campos mandatorios)	SÍ	Bootstrap 5 validation en incidencias.form.component.html con novalidate, was-validated, contadores de caracteres

Validaciones estrictas y sanitización en el Backend	PARCIAL	StoreIncidentRequest valida incident_category_id, location_id, priority, geom pero no valida titulo, descripcion, telefono
Historial y trazabilidad en los cambios de estado de la incidencia	SÍ	Trigger trg_log_incident_status registra automáticamente en tabla status_history con estado anterior/nuevo y timestamp
Roles y permisos granularmente definidos en la base de datos	PARCIAL	Las tablas permissions, role_permission existen con FKs y cascade; falta UI para gestionar la asignación granular de permisos por rol
Autenticación con token implementada	SÍ	JWT propio con refresh tokens, sesiones revocables almacenadas en BD
Soft deletes en entidades principales	SÍ	SoftDeletes en Incident, User, IncidentCategory, Organization
Triggers de base de datos para integridad	SÍ	3 triggers activos: validación categoría hoja, log de estados, asignación geográfica automática
Control de acceso por recurso (no solo por autenticación)	NO	El middleware autentica, pero no verifica ownership de incidencia por usuario
Políticas de complejidad de contraseña	NO	Solo min:8 — sin requerir mayúsculas, números ni símbolos
Rate limiting / Protección contra fuerza bruta	NO	El endpoint /api/login no tiene middleware throttle

6. Registro de Hallazgos Encontrados

ID	Hallazgo / Defecto Identificado	Severidad	Archivo / Evidencia
H-01	Ausencia de campos titulo y descripcion en la tabla incidents y en StoreIncidentRequest — el formulario frontend los captura pero el backend los descarta silenciosamente.	Alta	create_incidents_table.php, StoreIncidentRequest.php, incidencias.form.component.html
H-02	Endpoint POST /api/login sin middleware de rate-limiting — vulnerable a ataques de fuerza bruta sin bloqueo automático.	Alta	routes/api.php línea Route::post('/login', ...) — no tiene throttle:
H-03	El middleware JwtAuthenticate autentica usuario pero los	Alta	JwtAuthenticate.php, IncidentController.php — sin verificación de ownership

	controladores no verifican que el recurso (incidencia, comentario) pertenezca al usuario autenticado		
H-04	La contraseña solo exige mínimo 8 caracteres — sin política de complejidad (sin mayúsculas obligatorias, números o caracteres especiales)	Media	StoreUserRequest.php regla <code>`password'=> 'required'</code>
H-05	El modo APP_DEBUG=true expone stack traces completos con rutas de archivos del servidor en respuestas HTTP de error	Media	Configuración .env estándar de Laravel en desarrollo
H-06	El trigger <code>auto_assign_location()</code> falla silenciosamente si no hay geometrías (geom) en la tabla locations — la ubicación queda como NULL sin notificar al usuario	Baja	Función PL/pgSQL en <code>create_incident_triggers.php</code> — <code>IF v_location_id IS NOT NULL THEN</code> sin rama <code>ELSE</code> de error

7. Acciones Correctivas propuestas

ID Hallazgo	Plan de Acción / Resolución Técnica
H-01	Agregar columnas <code>titulo VARCHAR(100) NOT NULL</code> y <code>descripcion VARCHAR(500) NOT NULL</code> a la migración <code>create_incidents_table</code> (o nueva migración <code>add_title_to_incidents_table</code>). Actualizar <code>StoreIncidentRequest</code> con reglas <code>`titulo' => 'required'</code>
H-02	Agregar middleware <code>throttle:5,1</code> al grupo de rutas públicas de autenticación en <code>routes/api.php</code> : <code>Route::middleware('throttle:5,1')->group(function () { Route::post('/login', ...); });</code> . Esto limita a 5 intentos por minuto por IP.
H-03	Implementar Políticas de Laravel por recurso. En <code>IncidentController::show(), update(), destroy()</code> : verificar <code>\$this->authorize('view', \$incident)</code> . En <code>IncidentPolicy::view()</code> : retornar <code>`\$user->id === \$incident->user_id'</code>

H-04	Actualizar la regla de contraseña en StoreUserRequest a 'password' => ['required', 'string', 'min:8', 'regex:/[A-Z]/', 'regex:/[0-9]/'] y agregar mensajes de error descriptivos en messages().
H-05	Configurar en bootstrap/app.php el handler de excepciones para que en producción (APP_ENV=production) los errores retornen respuestas genéricas JSON sin exponer stack traces. Asegurarse que el .env de producción tenga APP_DEBUG=false.
H-06	Modificar la función auto_assign_location() para que si v_location_id IS NULL se haga RAISE WARNING 'No location found for geom: %', ST_AsText(NEW.geom) y/o se inserte un registro en una tabla de logs de advertencias geoespaciales para revisión posterior.